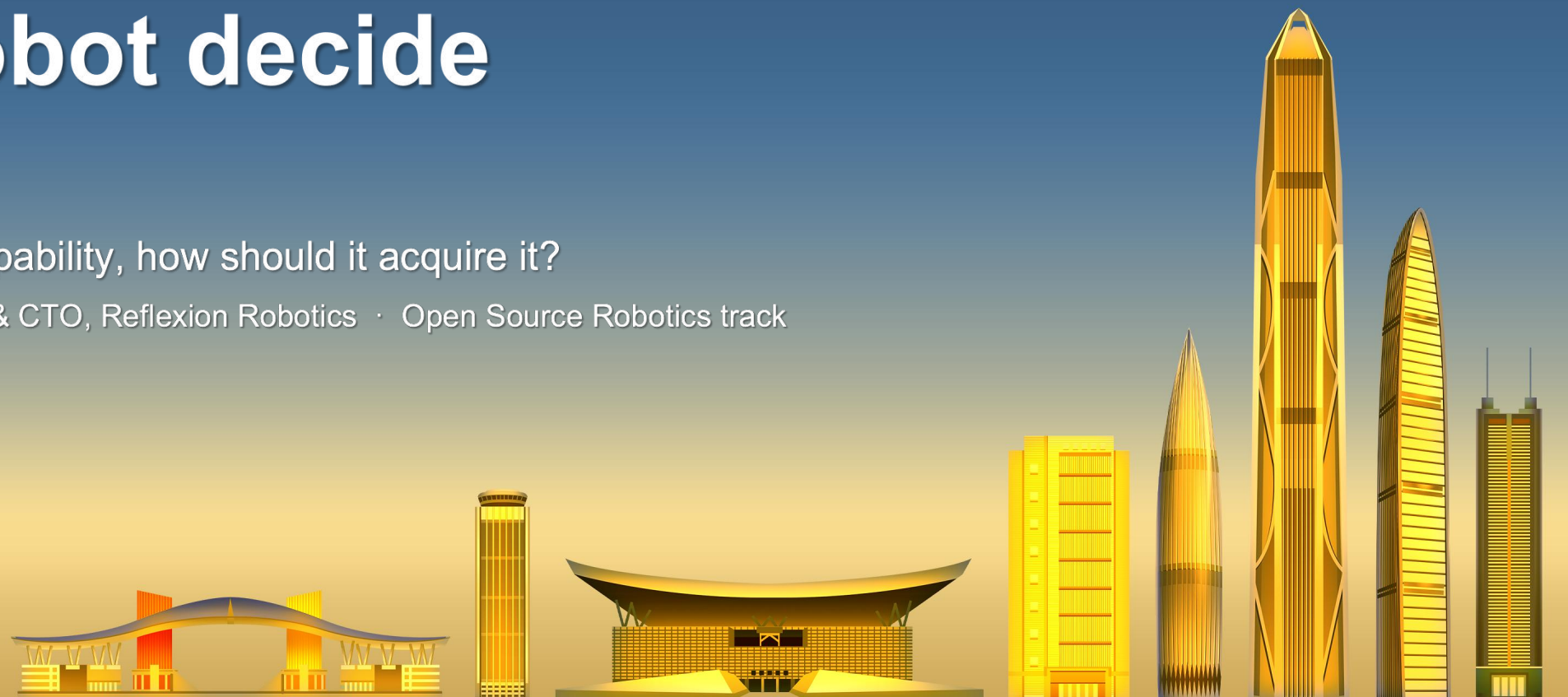
GOSIM

GOSIM Shenzhen 2026

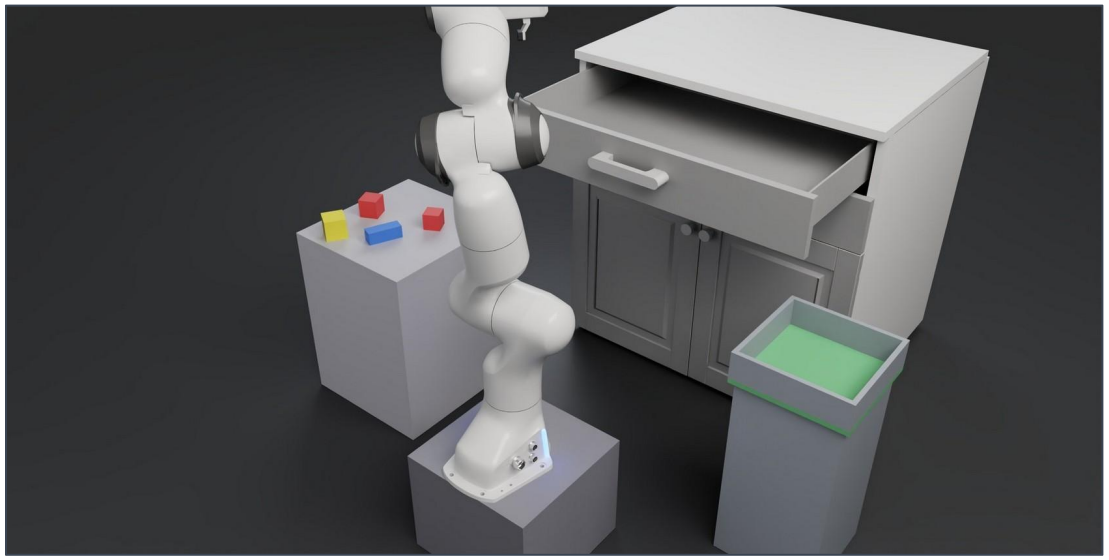
Train it or code it? Let the robot decide

When a robot lacks a capability, how should it acquire it?

Charbel Dandjinou · Founder & CTO, Reflexion Robotics · Open Source Robotics track



A failure is not a request for more training



A manipulation scene in Isaac Lab, used throughout as the running example

The arm reaches the drawer, but the part is not where it should be.

Was the call wrong?

→ a parameter or a program to fix

Was the scene disturbed or misread?

→ look again, then retry the same skill

Is the controller inadequate?

→ a policy to train or to replace

The same visible failure can require different interventions. Today a human engineer decides, usually before seeing the failure.

Code it or train it: both are right, for different failures

Code it

Minutes, not hours.

Readable, debuggable, reviewable.

Composes the primitives the API already exposes.

Limit: *a program can only compose the primitives its API exposes, so no amount of program repair invents a dynamic balance recovery.*

Train it

Discovers motor strategies nobody wrote.

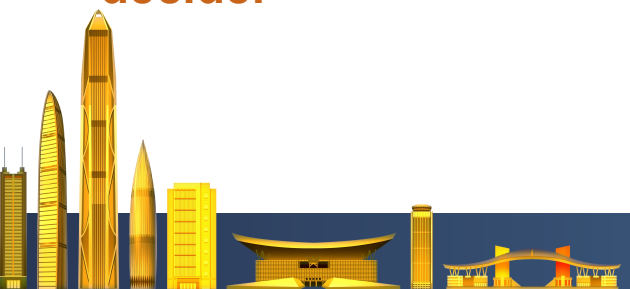
Handles contact, balance, recovery.

Costs a training run, a reward or demonstrations.

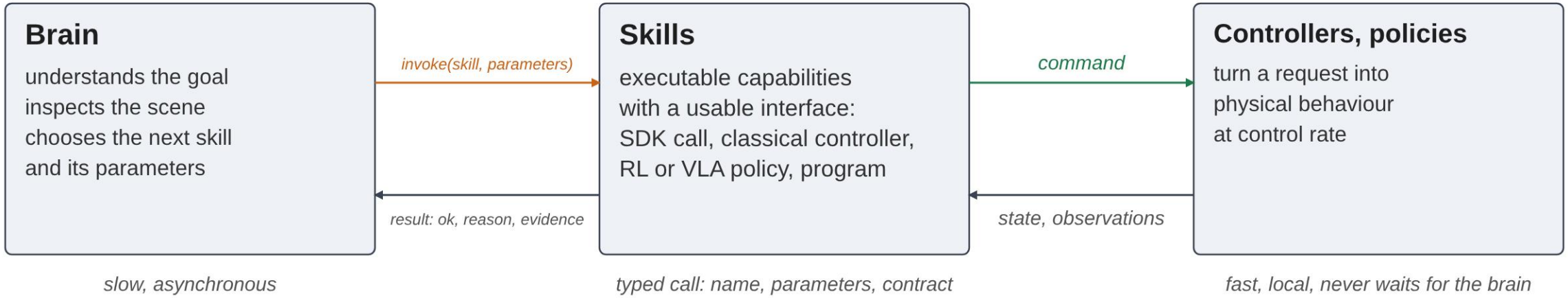
Limit: *a reward function is not worth a training run for ten lines of control code.*

Why now: the agent that writes the code is the agent that launches the training. For the first time both answers are reachable from the same place.

The right answer depends on what the failure looks like. Whoever sees the failure is best placed to decide.



The brain is not the motor policy



The brain needs to know three things about a skill:

What can it do? What does it need? Did it work?

Figure 1. A learned policy is one implementation of a skill, so choosing a model is no longer choosing the whole robot architecture (Dandjinou, TechRxiv 2025).

Skill Foundry: an agent that develops its own capabilities

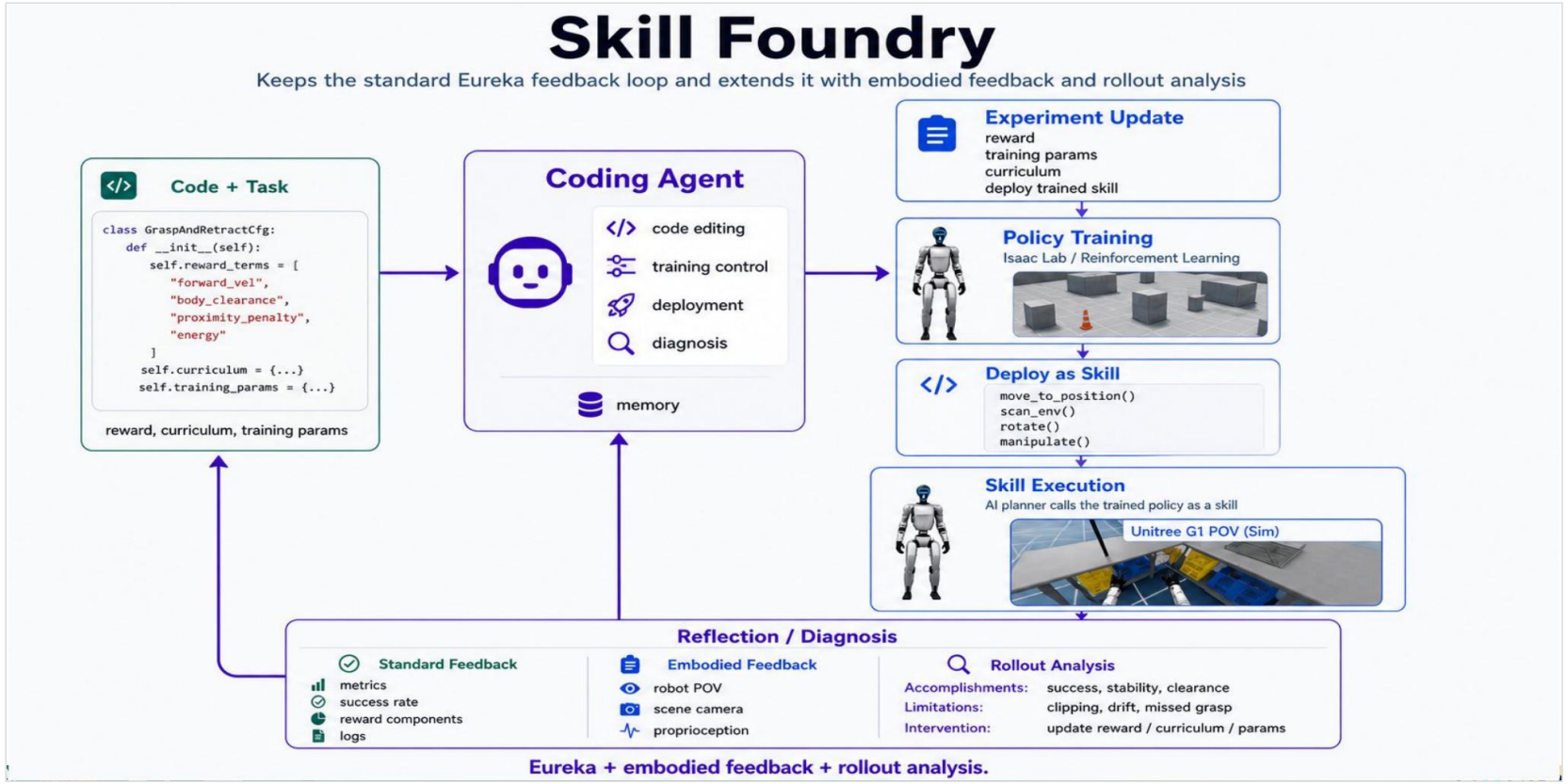


Figure 2. The Skill Foundry loop as presented at GOSIM Paris, 6 May 2026 (original slide 24): Eureka + embodied feedback + rollout analysis.

The robot is your body. Simulation is your training ground. Your mission is to evolve before entering the physical robot.

When you hit a limitation, you can code.

When a behaviour fails, you can train.

When a skill is missing, you can create it.

When a new skill overlaps, look for something orthogonal.

Architecture: Dandjinou, TechRxiv, 18 Nov 2025.
Loop: GOSIM Paris, May 2026.

The feedback must explain the intervention

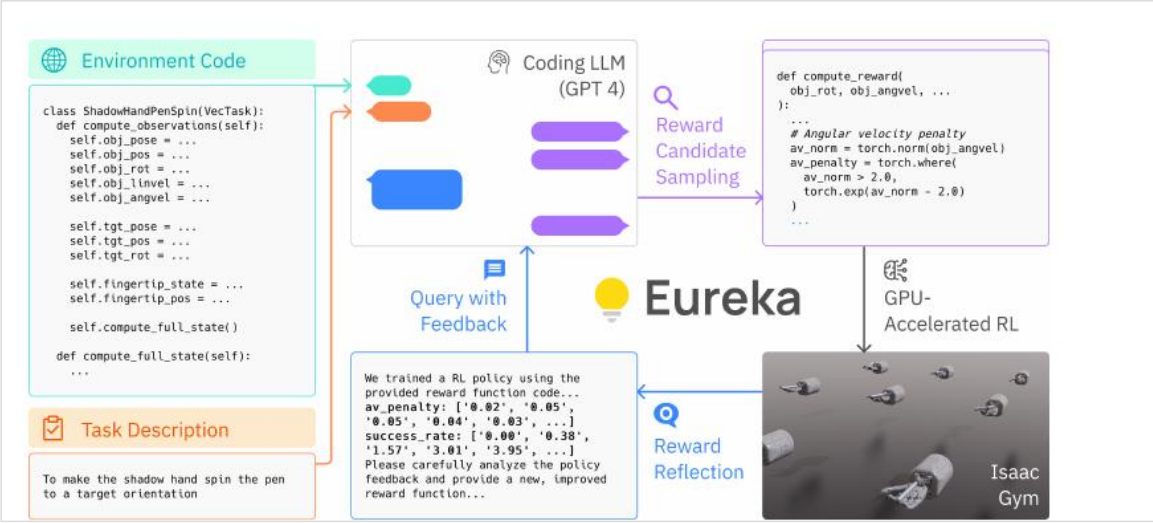


Figure 3. The Eureka loop (Ma et al., 2023, Fig. 2): the LLM writes the reward, RL trains, a scalar comes back.

Eureka showed that a language model searches over rewards faster than a human edits them. Its only feedback is a scalar: **a scalar says that something changed, not why.**

Metrics **what changed?**

success rate, reward components, collision count

Robot view and scene view **what happened?**

the shoulder clipped the obstacle, the turn came too late, the grasp slipped after contact

Robot state and execution trace **during which action?**

joint states, timestamps, the active skill and its parameters

A diagnosis says why, and already points at the intervention.

Code it: the program lineage

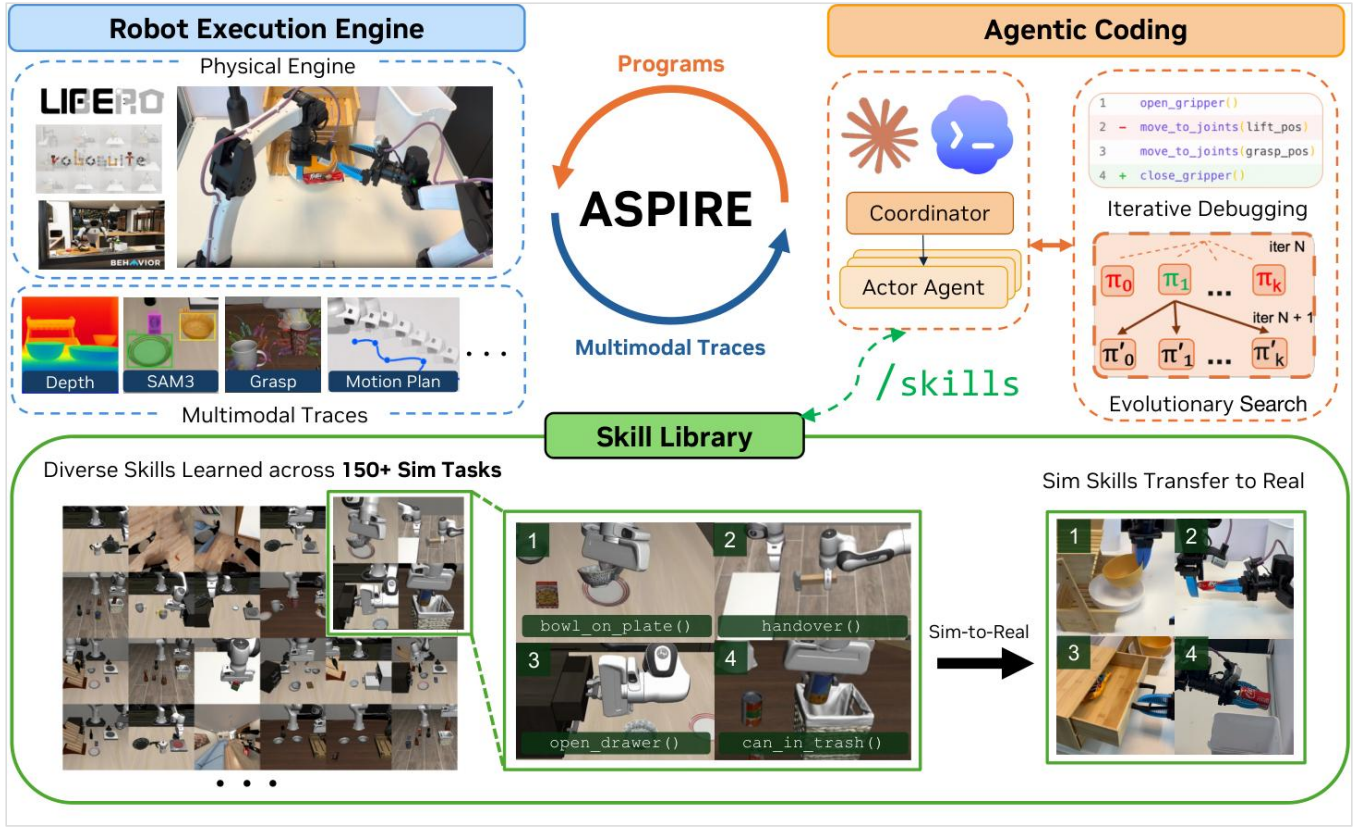


Figure 4. ASPIRE system overview (Lu et al., NVIDIA, CMU, UC Berkeley, UMich, 2026, Fig. 1).

Code as Policies Google, 2022

An LLM writes policy code that calls perception and control APIs. The generated program is the policy.

RHO UC Berkeley, AMD, June 2026

A coding agent searches over multi-file policy repositories with reward and execution feedback. The artifact is a code repository.

ASPIRE NVIDIA, CMU, UC Berkeley, UMich, June 2026

An agent writes and repairs control programs from closed-loop multimodal traces and keeps validated repairs as reusable guidance. Example: a navigation goal 20 cm from a table edge, fixed by a multi-angle approach routine.

The product is code. What stays fixed: the primitives the API exposes, a limit the ASPIRE authors state themselves. When the API has no primitive for the motion, repair cannot invent it.

Train it: the policy lineage

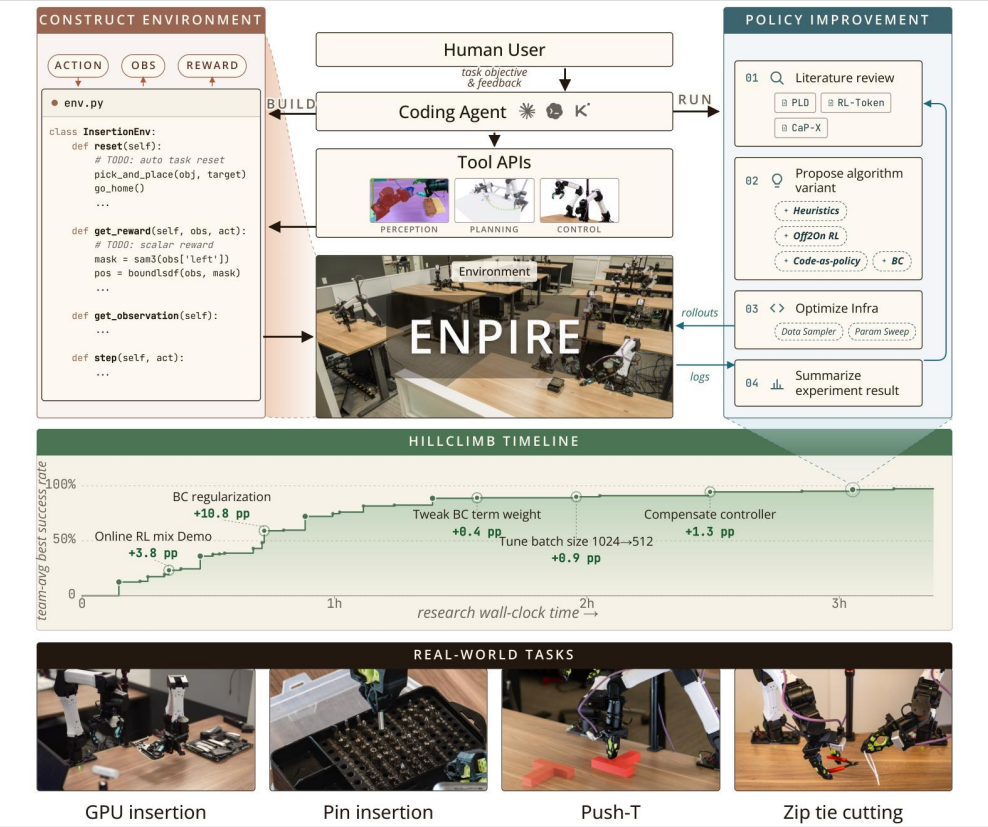


Figure 5. ENPIRE framework (Xiao et al., NVIDIA, CMU, UC Berkeley, 2026, Fig. 2).

Eureka NVIDIA, 2023

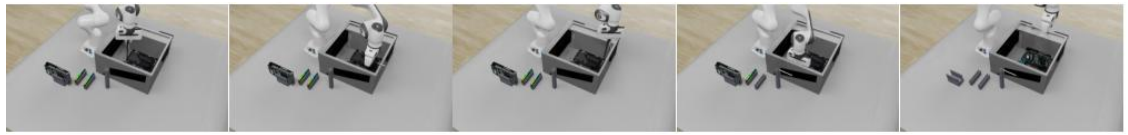
An LLM writes the reward, RL trains the policy, scalar metrics come back, the LLM edits the reward. Fast search over rewards, blind to what the robot did.

ENPIRE NVIDIA, CMU, UC Berkeley, June 2026

A harness for coding agents on real robots: reset, execute, verify, refine. The agent may pick heuristics, policy code, BC or RL for the task at hand, inside a loop whose product is that task's policy.

EmbodiedSWE Princeton, Stanford, Yale, CMU, Sept 2026

Coding agents solve long-horizon dexterous tasks in simulation; one verified program is diversified into demonstrations that fine-tune a VLA. Coding and training as successive stages.



EmbodiedSWE, PC assembly (Shen et al., 2026, Fig. 2): drive bolts, press-fit the RAM and GPU.

The product is weights. The loop runs offline, per task, before deployment.

Who answers "train it or code it?"

decided by the agent, at runtime, from the observed failure, across the repertoire

Skill Foundry
programs and policies behind one contract

the output is whatever the failure calls for: retry, compose, code or train

decided by the agent, offline, per task, inside a harness

ENPIRE
heuristics, code, BC or RL for one task's policy

decided by the authors, when the system is designed

Code as Policies, RHO
the product is code

ASPIRE
code and repair knowledge

EmbodiedSWE
code, then a VLA trained on it

Eureka
the product is a policy

Recova
learned recovery, or a human demo first

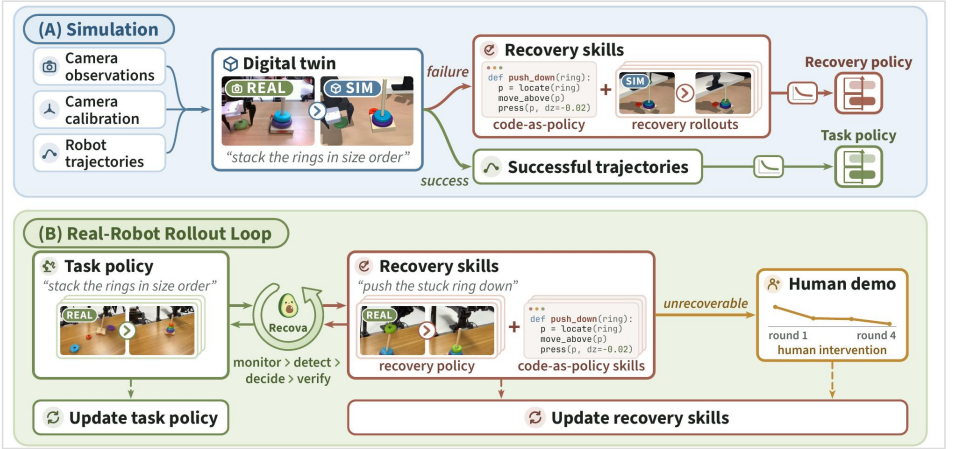


Figure 7. Recova: simulation and real-robot recovery loop (Liu et al., NVIDIA, UCSD, UT Austin, 2026, Fig. 2).

Recova holds both kinds of recovery: a learned policy when the failure is in its registry, otherwise a human demo first, then programs in a twin and a new trained skill. The order is fixed by the authors.

ENPIRE lets its agent pick heuristics, code, BC or RL, for one task, offline.

Skill Foundry takes the choice at runtime, from the failure, across the repertoire.

Nobody else decides at runtime, from the failure, across the repertoire: the position this talk defends and proposes to test.

Skill Foundry makes capability acquisition a decision

Reuse	Compose	Code	Train
the capability already exists	existing capabilities solve the problem together	new executable logic is the right intervention	learning or adapting a policy is the right intervention

Choose from the evidence, the repertoire and the budget.

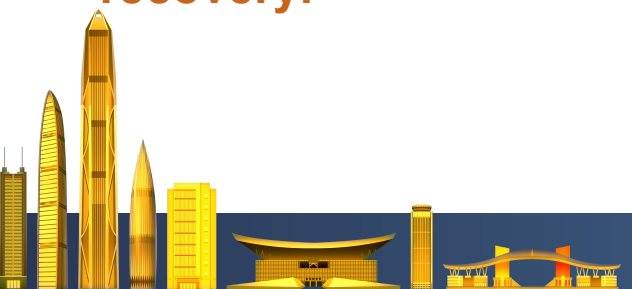
One contract for programs and trained policies. the result of a training run and the result of a code edit land in the same registry, callable the same way

Observation streamed during execution. a buffer the agent checks while the rollout runs, so the failure is located in time and a doomed rollout is cut short

Memory of attempts, including the failed ones. working, long-term, spatial and skill memory, so a rejected hypothesis changes the next experiment instead of being rediscovered

A coverage constraint on every new skill. a skill is accepted when it solves situations no existing skill covered

Four mechanisms make the choice possible at runtime, across the whole repertoire, not only for recovery.



Two loops, one evolving repertoire

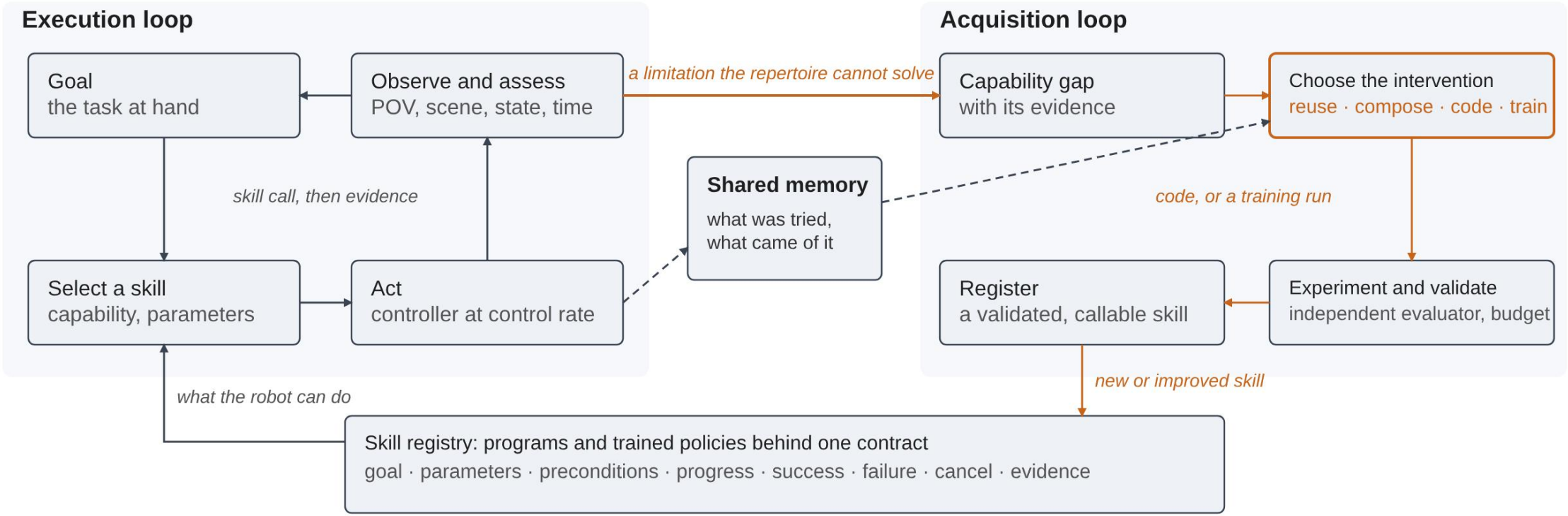
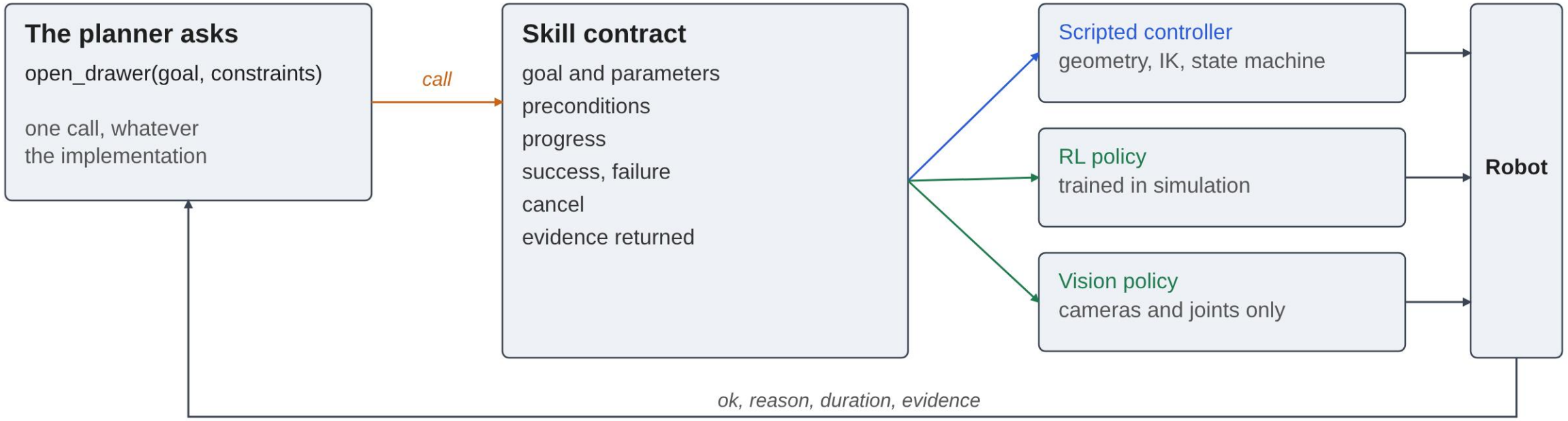


Figure 6. Two loops with different time scales: the controller keeps acting while the brain observes; a training run is a separate operation with a budget and a verdict.

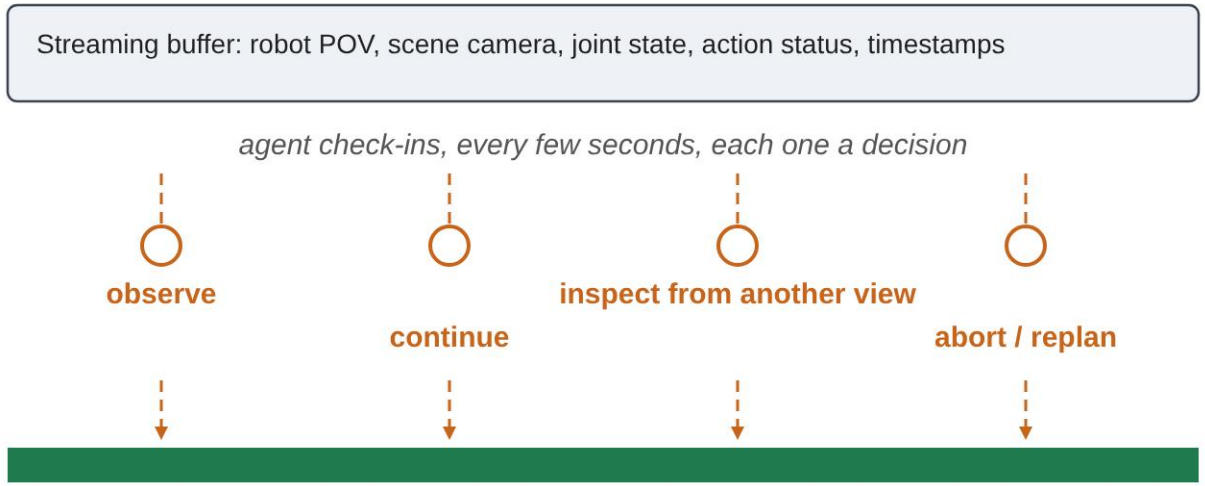
One call, several providers



A raw action stream is not automatically open_drawer: the provider must really hold the contract.

Figure 7. Where a skill came from is a property of the skill, not of the interface, so the result of a training run and the result of a code edit land in the same registry.

Observe while the robot acts





Insertion seen through low-resolution robot cameras: the last millimetres are invisible, so the agent must choose where to look before it judges.

Seeing well means choosing where to look.

Observation must be able to change the next action.

A doomed rollout is cut before it ends; on hardware, before damage; the failure is located in time.

Figure 8. Asynchronous supervision: the brain reads a time-stamped buffer while the controller runs; it never drives motor torques.

Choose the intervention that addresses the limitation

Evidence read from the execution	Candidate intervention	
The scene is ambiguous, or the evidence is stale	**Obtain another observation**	inspect
The skill is suitable, the call or the result was disturbed	**Correct the call, check the result, retry**	reuse
The primitives work, their coordination fails	**Compose the skills, or write explicit logic**	code
The current controller cannot produce the behaviour	**Train, adapt or replace the policy**	train

Compare expected improvement, cost and future reuse. Candidate interventions, not an automatic rule.

Figure 9. From evidence to intervention: the diagnosis comes first, the method follows.

The decision is written in the rollout, not in the team's habits: diagnose first, then pick the cheapest intervention that addresses the cause.

Grow coverage, not library size

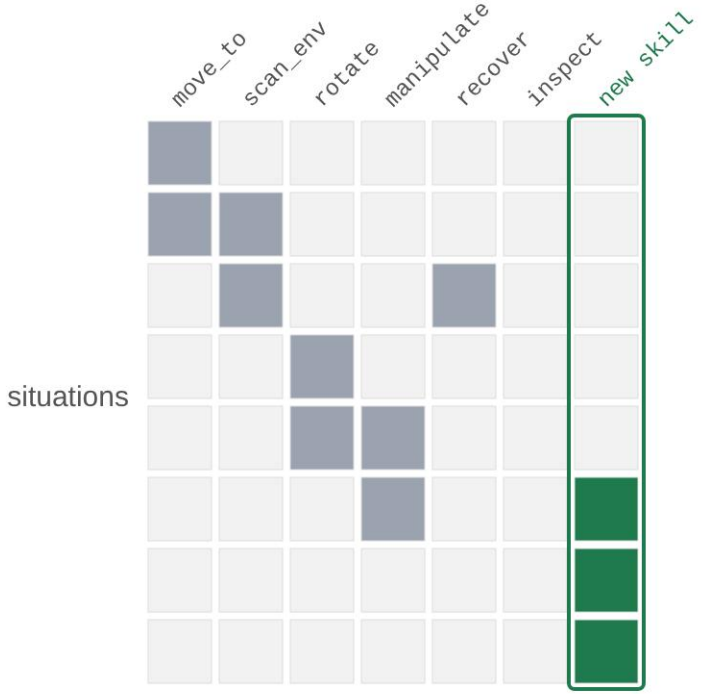


Figure 10. Coverage as the acceptance criterion for a new skill.

A new skill matters when it solves situations no existing skill covered.

Count what becomes solvable: tasks, conditions, failure modes.
Not the size of the library.

Orthogonality is a constraint given to the agent when it creates a skill,
not a pruning step afterwards.

A component that changes no decision adds no coverage and is removed.
Memory keeps the dead ends closed, so the library grows toward coverage.

Test the decision, not only the final success

Fixed code-first strategy

every failure is answered by a program edit

Fixed train-first strategy

every failure is answered by a training run

Adaptive strategy

the agent chooses from the evidence, the repertoire and the budget

Same starting capabilities, same observations, same task distribution.

Measure. task success; time and resources to acquire a capability (GPU hours, model calls, robot time); reuse on new situations

Protocol. paired runs with the same seeds, confidence intervals, an evaluator the agent never sees, a held-out set never used for tuning

Ablations. adaptive selector against each fixed strategy; memory on against off; observation during execution against final feedback only

Does adaptation improve the return on experimentation? This is the experiment, not a result.



Who decides what the robot learns next?

Policies make robots capable.

Agents can make their capabilities evolve.

Reuse · Compose · Code · Train

The next capability should determine the method, not the other way around.

charbel.dandjinou@reflexionrobotics.com

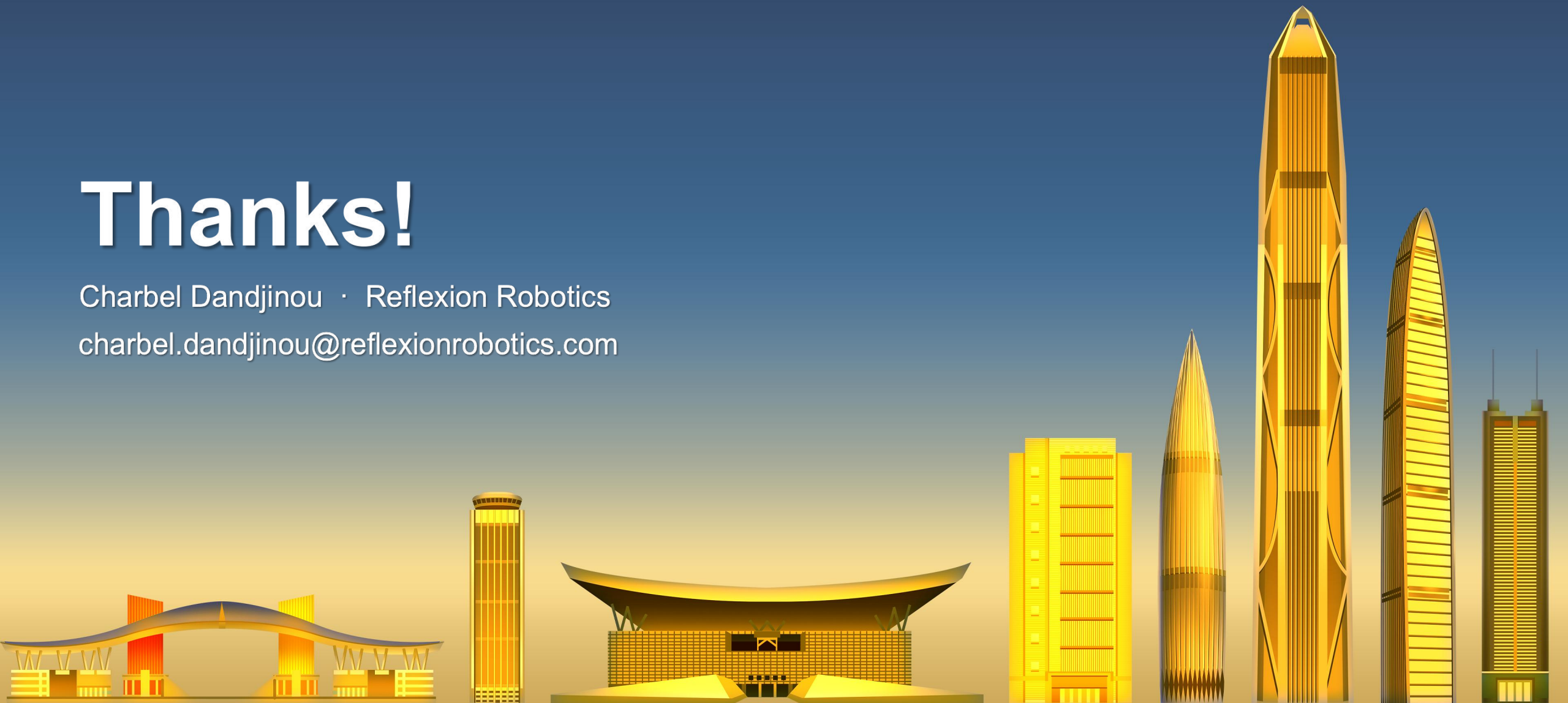


GOSIM

GOSIM Shenzhen 2026

Thanks!

Charbel Dandjinou · Reflexion Robotics
charbel.dandjinou@reflexionrobotics.com



References: foundations and public milestones

Dandjinou, A hierarchical architecture for embodied AI, TechRxiv, 18 Nov 2025

doi.org/10.36227/techrxiv.176344191.17191825/v1

Skill Foundry, GOSIM Paris, 6 May 2026

paris2026.gosim.org

Liang et al., Code as Policies, 2022

arxiv.org/abs/2209.07753

Ma et al., Eureka, 2023

arxiv.org/abs/2310.12931

Ahn et al., SayCan, CoRL 2022

proceedings.mlr.press/v205/ichter23a.html

Huang et al., Inner Monologue, 2022

proceedings.mlr.press/v205/huang23c.html

References: the 2026 agentic wave

Elmaaroufi et al., RHO: Your Coding Agent is Secretly a Roboticist, 15 Jun 2026
arxiv.org/abs/2606.16458

Xiao et al., ENPIRE: Agentic Robot Policy Self-Improvement in the Real World, 18 Jun 2026
arxiv.org/abs/2606.19980

Lu et al., ASPIRE: Agentic Skills Discovery for Robotics, 30 Jun 2026
arxiv.org/abs/2607.00272

Shen et al., EmbodiedSWE: Coding Agents for Long Horizon Dexterous Robotics, 23 Sep 2026
arxiv.org/abs/2609.27308

Liu et al., Recova: Agent-Guided Failure Recovery for Autonomous Robotic Manipulation, 1 Oct 2026
arxiv.org/abs/2610.01178

